Chapter 8: Data Flow Analysis

Download this chapter as a PDF for offline reading:
Chapter-08-Data-Flow-Analysis.pdf

Matching syntactic patterns finds some bugs, but many vulnerabilities only appear when you track how values move through a program. Data-flow analysis answers the question "can this value reach that operation?" This chapter introduces local and global data flow, shows how to define a configuration, and covers the two most common ways to refine a configuration: additional flow steps and barriers.

Local and Global Flow

CodeQL models two kinds of data flow.

Kind	Scope	Precision	Cost
Local	Within a single function	Higher	Lower
Global	Across the whole program (interprocedural)	Lower	Higher

Prefer local flow when it is sufficient. Reach for global flow only when the interesting path crosses function boundaries.

Local data flow

Local flow tracks values inside one function. The predicate `DataFlow::localFlow(source, sink)` is true when there is a path from `source` to `sink` that stays inside the same function.

Example that finds values flowing into `eval` within a single function (see `examples/chapter08/local-flow-to-eval.ql`):

```
/**
 * @name Local flow into eval
 * @description Finds values that flow into a call to eval within the same function.
 * @kind problem
 * @id mustlearn/js-local-flow-to-eval
 * @tags security
 * @problem.severity warning
 * @precision high
 */
import javascript

from DataFlow::Node source, DataFlow::Node sink
where
  sink = any(CallExpr c | c.getCalleeName() = "eval").getArgument(0).flow() and
```

```
  DataFlow::localFlow(source, sink)
select sink, "Value flows into eval: " + source
```

Local flow is fast enough that you can compute it for every function in a database.

Global data flow

Global flow tracks values across function calls, returns, and (in some languages) object fields. It is expressed with a configuration module that implements `DataFlow::ConfigSig` (or the older class-based `Configuration` style). You then instantiate the global analysis:

```
module Flow = DataFlow::Global<FlowConfig>;
```

The analysis is queried with `Flow::flow(source, sink)` or, for path queries, `Flow::flowPath(source, sink)`.

Configurations, Additional Steps, and Barriers

A configuration defines three things:

- **Sources** — where interesting values originate (`isSource`).
- **Sinks** — where those values become dangerous or interesting (`isSink`).
- **Optional refinements** — extra steps that the default model misses (`isAdditionalFlowStep`) and places where flow should stop (`isBarrier` or `isSanitizer`).

Minimal global configuration

Skeleton (see `examples/chapter08/global-flow-skeleton.ql`):

```
import javascript
import DataFlow::PathGraph

module FlowConfig implements DataFlow::ConfigSig {
  predicate isSource(DataFlow::Node source) {
    source instanceof RemoteFlowSource
  }

  predicate isSink(DataFlow::Node sink) {
    sink = any(CallExpr c | c.getCalleeName() = "eval").getArgument(0).flow()
  }
}

module Flow = DataFlow::Global<FlowConfig>;

from Flow::PathNode source, Flow::PathNode sink
where Flow::flowPath(source, sink)
select sink.getNode(), source, sink, "Flow from remote source to eval"
```

This is a path-problem query, so it needs the four-column select and the path-graph import (as introduced in Chapter 6).

Additional flow steps

Sometimes the default model does not know that a value moves from one node to another (for example, a custom getter, a library call whose body is not in the database, or an unusual language feature). You add the missing edge with `isAdditionalFlowStep`:

```
predicate isAdditionalFlowStep(DataFlow::Node pred, DataFlow::Node succ) {
  // Example: flow from the argument of myGet() to its return value
  exists(CallExpr c |
    c.getCalleeName() = "myGet" and
    pred = c.getArgument(0).flow() and
    succ = c.flow()
  )
}
```

Use additional steps sparingly; each extra edge increases the size of the search space.

Barriers

A barrier stops flow at a particular node. Typical uses are sanitizers, type checks that prove a value is safe, or simply places you do not care about:

```
predicate isBarrier(DataFlow::Node node) {
  node = any(CallExpr c | c.getCalleeName() = "sanitize").getArgument(0).flow()
}
```

In taint-tracking configurations the corresponding predicate is often called `isSanitizer` (covered in Chapter 9).

Exercises

Suggested exercises are in `examples/chapter08/EXERCISES.md`:

1. Write a local data-flow query that finds string literals flowing into `console.log` within the same function.
2. Extend the global-flow skeleton so that it only reports flows that pass through a function parameter.
3. Add an `isBarrier` that blocks flow through a sanitizer named `sanitize`.
4. (Optional) Add an `isAdditionalFlowStep` that models flow from the return value of `getUserInput` into its callers.

Next Steps

You can now express both local and global data-flow questions and refine them with barriers and extra steps. The next chapter turns to taint tracking—the variant of data flow that also follows values through operations that do not preserve the original value (concatenation, arithmetic, container operations, and so on)—and shows how to present those results as path queries.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter08/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL